# SaveDialog UI仕様書

## 1. 概要

### 1.1 コンポーネントの目的

SaveDialogは、Petri Net編集画面でユーザーが編集中のワークフローをDraftとして保存する際に表示されるモーダルダイアログコンポーネントです。Draft名（必須）と説明（オプション）を入力し、データベースに保存します。

**主な機能:**

- Draft名と説明の入力フォーム
- バリデーション機能（必須チェック、文字数制限、空白チェック）
- 保存処理中のローディング表示
- エラーメッセージ表示
- 保存/キャンセル操作

### 1.2 使用場面

- **表示タイミング**:
  - Petri Net編集画面（PetriNetEditorInner）で「保存」ボタンをクリックした時
  - ツールバー（EditorToolbar）の保存ボタン（SaveOutlined）をクリックした時
- **表示条件**: ユーザーが編集内容を保存する意図を示した時
- **操作**: Draft名と説明を入力して「保存」ボタンをクリック、またはキャンセル

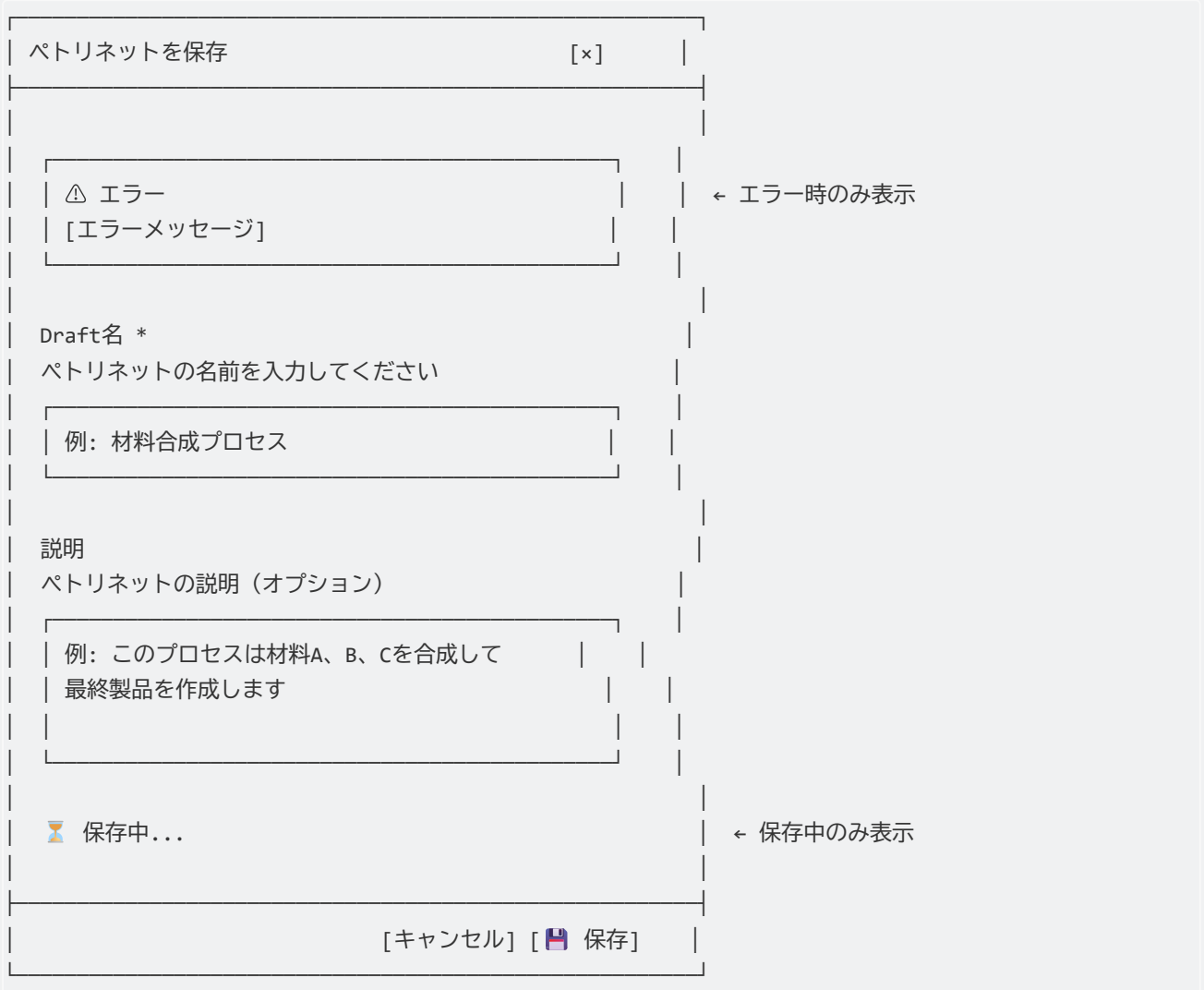
### 1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証
- **テナント分離**: organization_idによる分離

---

# 2. 画面構成

## 2.1 レイアウト構造




## 2.2 主要UI要素

### 2.2.1 モーダルダイアログ

- **タイトル**: "ペトリネットを保存"
- **幅**: 600px
- **マスククリック**: 保存中は無効化
- **キーボードESC**: 保存中は無効化

- **閉じるボタン**: 右上に×ボタン

### 2.2.2 エラー表示（Alert）

- **表示条件**: `error` propsが存在する場合
- **タイプ**: error（赤）
- **アイコン**: 警告アイコン（⚠）
- **閉じるボタン**: あり（×）
- **メッセージ**: "エラー"
- **説明**: `error` propsの内容

### 2.2.3 Draft名入力フィールド

- **ラベル**: "Draft名" + 必須マーク（*）
- **ヘルプテキスト**: "ペトリネットの名前を入力してください"
- **プレースホルダー**: "例: 材料合成プロセス"
- **必須**: はい
- **最大文字数**: 100文字
- **バリデーション**:
  - 必須チェック
  - 空白のみの入力を拒否
  - 100文字以内
- **オートフォーカス**: ダイアログ表示時に自動フォーカス
- **Enterキー**: 保存処理を実行（保存中は無効）

### 2.2.4 説明入力フィールド

- **ラベル**: "説明"
- **ヘルプテキスト**: "ペトリネットの説明（オプション）"
- **プレースホルダー**: "例: このプロセスは材料A、B、Cを合成して最終製品を作成します"
- **必須**: いいえ
- **行数**: 4行
- **最大文字数**: 制限なし

- **コンポーネント**: TextArea

### 2.2.5 ローディング表示

- **表示条件**: `isLoading === true`
- **アイコン**: Spin（小サイズ）
- **テキスト**: "保存中..."
- **色**: rgba(0, 0, 0, 0.45)
- **配置**: 左寄せ

### 2.2.6 フッターボタン

- **キャンセルボタン**:
  - ラベル: "キャンセル"
  - タイプ: default
  - 無効化: 保存中（`isLoading === true`）
- **保存ボタン**:
  - ラベル: "保存" / "保存中..."（状態によって切り替え）
  - タイプ: primary
  - アイコン: SaveOutlined（💾）/ Spin（保存中）
  - 無効化: 保存中（`isLoading === true`）

---

# 3. 機能仕様

## 3.1 ダイアログの表示・非表示

### 3.1.1 表示処理

**処理フロー:**

```
// ダイアログが開かれたとき
useEffect(() => {
  if (open) {
    // フォームに初期値をセット
    form.setFieldsValue({
      name: initialName,
      description: initialDescription,
    });
  }
}, [open, initialName, initialDescription, form]);
```

**UI動作:**

- `open === true` になるとモーダルが表示される
- 初期値（`initialName`, `initialDescription`）がフォームにセットされる
- Draft名フィールドに自動フォーカス

### 3.1.2 非表示処理

**キャンセルハンドラー:**

```
const handleCancel = () => {
  if (!isLoading) {
    onClose();
  }
};
```

**UI動作:**

- 保存中でない場合のみ、`onClose()` が呼ばれる
- モーダルが閉じられる
- フォームの内容はリセットされない（次回表示時に初期値が再セットされる）

## 3.2 保存処理

### 3.2.1 保存ハンドラー

**処理フロー:**

```
const handleSave = async () => {
  try {
    // 1. フォームバリデーション
    const values = await form.validateFields();

    // 2. データを整形（トリム処理）
    const saveData = {
      name: values.name.trim(),
      description: (values.description || '').trim(),
    };

    // 3. 親コンポーネントのonSaveを呼び出し
    onSave(saveData);
  } catch (error) {
    console.error('Validation failed:', error);
  }
};
```

**UI動作:**

1. フォームのバリデーションを実行
2. バリデーションが成功したら、`onSave()` propsを呼び出し
3. バリデーションが失敗したら、エラーメッセージを表示（Formが自動処理）
4. `onSave()` 実行後の処理（ローディング、成功/失敗）は親コンポーネントが管理

## 3.2.2 Enterキー押下

**処理:**

```
<Input
  onPressEnter={(e) => {
    e.preventDefault();
    if (!isLoading) {
      handleSave();
    }
  }}
/>
```

**UI動作:**

- Draft名フィールドでEnterキーを押すと保存処理を実行
- 保存中は無視される
- デフォルト動作（フォーム送信）は抑制される

## 3.3 バリデーション

### 3.3.1 Draft名のバリデーションルール

```
rules={[
  // 必須チェック
  { required: true, message: 'Draft名は必須です' },

  // 文字数制限
  { max: 100, message: 'Draft名は100文字以内にしてください' },

  // カスタムバリデーション（空白のみを拒否）
  {
    validator: (_, value) => {
      if (value && value.trim() === '') {
        return Promise.reject(new Error('Draft名は必須です'));
      }
      return Promise.resolve();
    },
  },
]}
```

**バリデーションケース:**

| ケース | 入力値 | 結果 | エラーメッセージ |
|---|---|---|---|
| 空文字 | "" | ❌ NG | "Draft名は必須です" |
| 空白のみ | " " | ❌ NG | "Draft名は必須です" |
| 101文字 | "a" × 101 | ❌ NG | "Draft名は100文字以内にしてください" |
| 正常 | "材料合成プロセス" | ✅ OK | - |

### 3.3.2 説明のバリデーション

**ルール:**

- オプション（バリデーションなし）

- 文字数制限なし
- 空文字列も許可

## 3.4 ローディング状態

### 3.4.1 ローディング中のUI

**表示要素:**

- ローディングインジケータ（Spin）
- "保存中..." テキスト
- 保存ボタンのローディング状態

**無効化要素:**

- Draft名入力フィールド
- 説明入力フィールド
- キャンセルボタン
- 保存ボタン
- マスククリック
- ESCキー

**制御:**

```
disabled={isLoading}
loading={isLoading}
maskClosable={!isLoading}
keyboard={!isLoading}
```

## 3.5 エラー処理

### 3.5.1 エラー表示

**条件:**

- `error` propsが存在する場合（空文字列でない）

**UI動作:**

- フォームの上部にエラーAlertを表示
- 赤色の背景、警告アイコン
- エラーメッセージを表示
- 閉じるボタン（×）で非表示にできる

**表示例:**

```
⚠ エラー
保存に失敗しました：サーバーエラー
```

---

# 4. データ仕様

## 4.1 Props定義

```typescript
export interface SaveDialogProps {
  open: boolean;                    // ダイアログの表示/非表示
  onClose: () => void;              // 閉じる時のコールバック
  onSave: (data: SaveData) => void; // 保存時のコールバック
  isLoading?: boolean;              // ローディング状態
  error?: string;                   // エラーメッセージ
  initialName?: string;             // Draft名の初期値
  initialDescription?: string;      // 説明の初期値
}
```

**Props説明:**

| Prop名 | 型 | 必須 | デフォルト値 | 説明 |
|---|---|---|---|---|
| open | boolean | ✅ | - | ダイアログの表示状態（trueで表示、falseで非表示） |
| onClose | () => void | ✅ | - | ダイアログを閉じる時に呼ばれるコールバック |
| onSave | (data: SaveData) => void | ✅ | - | 保存ボタンクリック時に呼ばれるコールバック |

| | | | | |
|---|---|---|---|---|
| isLoading | boolean | - | false | 保存処理中かどうか（trueの場合、UIを無効化） |
| error | string | - | undefined | エラーメッセージ（存在する場合、Alertで表示） |
| initialName | string | - | "" | Draft名の初期値 |
| initialDescription | string | - | "" | 説明の初期値 |

## 4.2 SaveData型定義

```
export interface SaveData {
  name: string;         // Draft名（トリム済み）
  description: string;  // 説明（トリム済み）
}
```

**SaveData説明:**

| フィールド名 | 型 | 必須 | 説明 |
|---|---|---|---|
| name | string | ✅ | Draft名（トリム処理済み、空白除去） |
| description | string | ✅ | 説明（トリム処理済み、空文字列も含む） |

## 4.3 State定義

```
// Ant Design Formインスタンス
const [form] = Form.useForm();
```

**State説明:**

| State名 | 型 | 説明 |
|---|---|---|
| form | FormInstance | Ant Design FormのAPI（setFieldsValue, validateFields等） |

## 4.4 フォームフィールド定義

```
interface FormValues {
  name: string;         // Draft名
```

```
  description: string;  // 説明
}
```

---

# 5. API連携

## 5.1 使用するAPI

SaveDialogコンポーネント自体はAPIを直接呼び出しません。

**データフロー:**

- `onSave(data: SaveData)` propsを呼び出し
- 親コンポーネント（PetriNetEditorInner）がAPI呼び出しを実行
- API呼び出し結果（成功/失敗）は親が `isLoading` と `error` propsで制御

## 5.2 データフロー図

```
SaveDialog
  ↓ handleSave()
onSave(data: SaveData)
  ↓
PetriNetEditorInner
  ↓ API呼び出し
POST /api/v1/experiment-admin/maiml/save-draft
  ↓ 成功
isLoading: false, error: undefined
  → ダイアログを閉じる

  ↓ 失敗
isLoading: false, error: "エラーメッセージ"
  → エラー表示
```

---

# 6. エラーハンドリング

# 6.1 エラーケース一覧

| エラーケース | 発生タイミング | 対処方法 |
| --- | --- | --- |
| バリデーションエラー | 保存ボタンクリック時 | Formが自動的にエラーメッセージを表示 |
| API保存エラー | 親コンポーネントのAPI呼び出し失敗 | `error` propsで受け取り、Alertで表示 |
| ネットワークエラー | API呼び出し中の通信エラー | 親コンポーネントが `error` propsで通知 |

# 6.2 エラー処理コード

## 6.2.1 バリデーションエラー

```
const handleSave = async () => {
  try {
    const values = await form.validateFields();
    onSave({
      name: values.name.trim(),
      description: (values.description || '').trim(),
    });
  } catch (error) {
    // バリデーションエラー
    // Formが自動的にエラーメッセージを表示するため、ここでは何もしない
    console.error('Validation failed:', error);
  }
};
```

## 6.2.2 API保存エラー

```
// 親コンポーネント（PetriNetEditorInner）で実装
const handleSaveDraft = async (data: SaveData) => {
  setIsLoading(true);
  setError(undefined);

  try {
    await apiService.saveDraft(data);
    message.success('Draftを保存しました');
    setSaveDialogOpen(false);
  } catch (error) {
    setError('保存に失敗しました: ' + error.message);
```

```
  } finally {
    setIsLoading(false);
  }
};
```

---

# 7. 実装ファイル

## 7.1 ファイルパス

- **メインファイル**: `frontend/src/components/editorv2/components/SaveDialog.tsx`

## 7.2 関連ファイル

- **型定義**: `frontend/src/types/editor.ts` - `SaveData` 型定義
- **親コンポーネント**: `frontend/src/components/editorv2/PetriNetEditorInner.tsx` - SaveDialogを呼び出す

## 7.3 依存コンポーネント

**Ant Designコンポーネント:**

- `Modal` - モーダルダイアログ
- `Form` - フォーム管理
- `Form.Item` - フォームフィールド
- `Input` - テキスト入力
- `Input.TextArea` - 複数行テキスト入力
- `Button` - ボタン
- `Alert` - エラー表示
- `Spin` - ローディングインジケータ

- `Space` - レイアウトコンポーネント

**Ant Designアイコン:**

- `SaveOutlined` - 保存アイコン（💾）

## 7.4 使用例

```
// PetriNetEditorInner.tsx
const [saveDialogOpen, setSaveDialogOpen] = useState(false);
const [saveLoading, setSaveLoading] = useState(false);
const [saveError, setSaveError] = useState<string | undefined>();

const handleSaveDraft = async (data: SaveData) => {
  setSaveLoading(true);
  setSaveError(undefined);

  try {
    await apiService.saveDraft({
      organizationId,
      projectName,
      draftName: data.name,
      description: data.description,
      maimlContent: getCurrentMaimlContent(),
    });

    message.success('Draftを保存しました');
    setSaveDialogOpen(false);
  } catch (error) {
    setSaveError('保存に失敗しました: ' + error.message);
  } finally {
    setSaveLoading(false);
  }
};

return (
  <>
    <SaveDialog
      open={saveDialogOpen}
      onClose={() => setSaveDialogOpen(false)}
      onSave={handleSaveDraft}
      isLoading={saveLoading}
      error={saveError}
      initialName=""
      initialDescription=""
    />
```

```
  </>
);
```

---

# 8. 補足情報

## 8.1 Material-UI → Ant Design変換

このコンポーネントは、Material-UIからAnt Designに変換されたバージョンです。

**主な変更点:**

- Dialog → Modal
- TextField → Input, TextArea
- CircularProgress → Spin
- DialogTitle, DialogContent, DialogActions → Modal構造

## 8.2 バリデーションのタイミング

**リアルタイムバリデーション:**

- フィールドからフォーカスが外れた時（onBlur）
- 入力値が変更された時（onChange）

**保存時バリデーション:**

- 保存ボタンクリック時
- Enterキー押下時

## 8.3 データのトリム処理

```
onSave({
  name: values.name.trim(),                    // 前後の空白を除去
  description: (values.description || '').trim(), // 説明も同様にトリム
});
```

**目的:**

- 意図しない空白の保存を防ぐ
- データベースの容量を節約
- 検索・表示時の一貫性を保つ

## 8.4 フォームのリセット

**現在の実装:**

- ダイアログを閉じてもフォームはリセットされない
- 次回表示時に初期値（`initialName`, `initialDescription`）が再セットされる

**利点:**

- ユーザーが誤ってダイアログを閉じた場合、入力内容が保持される
- 親コンポーネントが明示的に初期値を制御できる

## 8.5 保存ボタンのローディング表示

```
<Button
  key="save"
  type="primary"
  onClick={handleSave}
  disabled={isLoading}
  icon={!isLoading ? <SaveOutlined /> : undefined}
  loading={isLoading}
>
  {isLoading ? '保存中...' : '保存'}
</Button>
```

**UI動作:**

- 保存中: アイコンがSpinに変わり、ボタンが無効化され、ラベルが「保存中...」に変わる
- 通常時: SaveOutlinedアイコンが表示され、ラベルが「保存」

---

# 9. 変更履歴

| 日付 | 変更内容 | 担当者 |
| --- | --- | --- |
| 2026-03-12 | 実装ベースでUI仕様書を作成 | Cline |
| 2026-01-29 | Material-UIからAnt Designに変換（v2.0.0） | 開発チーム |